Babu Banarasi Das University

Case Study
ON

Movie Rating Analysis using Hadoop

Submitted To:-
Mr. Vikash (IBM)

Submitted By:-
Name – Ajay Chauhan
Section – BCADS21
Roll Number – 1240258047

PROJECT

Movie Dataset MapReduce Practical

- **Architecture:**

  - **The architecture of this project follows a standard Big Data pipeline using Hadoop ecosystem components.**

- **Data Flow:**

  - **Raw Dataset → HDFS → MapReduce Processing → Processed Data → Hive Analysis → Insights**

- **Explanation:**

  - **Raw dataset (movie data) is first stored in HDFS**
  - **MapReduce is used to process the data and calculate average ratings**
  - **Processed data is stored back in HDFS**
  - **Hive is used to perform analytical queries**
  - **Final insights are generated from the results**

- **Tools Used:**

  - **Hadoop Distributed File System (HDFS)**
  - **MapReduce (Hadoop Streaming using Python)**
  - **Hive (for data analysis)**
  - **Linux Terminal**
  - **ChatGPT (Generative AI tool)**

- **Technologies:**

  - **Python (for mapper and reducer)**
  - **CSV dataset**
  - **Hadoop Ecosystem**

Ajay Chauhan

➢ Use of Generative AI in the Project:

- **In this project, Generative AI tools were used to assist in development and understanding of Big Data concepts.**

➢ Areas Where AI Was Used:

1. Code Generation

- **Generative AI was used to generate MapReduce programs (mapper and reducer).**

2. Debugging

- **Errors in code were identified and fixed using AI assistance.**

3. Query Writing

- **Hive queries were generated and optimized using AI.**

4. Concept Understanding

- **AI helped in understanding:**
  - **HDFS architecture**
  - **MapReduce workflow**
  - **Hive operations**

➢ Example Prompt Used:

- **Write a Python MapReduce program to calculate average movie ratings.**

➢ Benefits of Using AI:

- **Faster development**
- **Reduced coding errors**
- **Better understanding of concepts**
- **Improved efficiency**

Ajay Chauhan

➢ Validation:

- **All AI-generated code was manually tested and verified before execution.**

➢ Dataset Description:

- **The dataset used in this project is a movie dataset containing user ratings and movie details. The data is stored in CSV (Comma Separated Values) format and is used for analysis using Hadoop.**

➢ Dataset Structure:

- **The dataset consists of the following columns:**
    - **userId – Unique ID of user**
    - **movieId – Unique ID of movie**
    - **title – Name of the movie**
    - **genre – Category of the movie**
    - **rating – User rating (1 to 5)**
    - **year – Release year of movie**
    - **watch_time(min) – Duration of movie**
    - **device – Device used for watching**
    - **location – Country of user**
    - **review_score – Detailed rating (1 to 10)**

➢ Dataset Source:

- **The dataset is a combination of real-world movie data and custom entries created for project demonstration.**

➢ Purpose of Dataset:

- **The dataset is used to:**
    - **Analyze movie ratings**
    - **Find top-rated movies**
    - **Perform data aggregation using MapReduce**
    - **Generate insights using Hive**

Ajay Chauhan

➢ Sample Data:

```
userId,movieId,title,genre,rating,year,watch_time(min),device,location,review_score
1,101,KGF,Action,5,2018,150,Mobile,India,9
2,102,RRR,Action,4,2022,180,Laptop,India,8
3,103,Inception,Sci-Fi,5,2010,148,TV,USA,9
4,104,Avengers,Action,4,2012,143,Mobile,UK,8
5,105,Titanic,Romance,5,1997,195,TV,Canada,9
6,101,KGF,Action,3,2018,150,Laptop,India,7
7,102,RRR,Action,5,2022,180,TV,India,9
8,103,Inception,Sci-Fi,4,2010,148,Mobile,USA,8
9,104,Avengers,Action,5,2012,143,Laptop,UK,9
10,105,Titanic,Romance,4,1997,195,TV,Canada,8
```

➢ Conclusion:

- **In this project, a complete Big Data pipeline was successfully implemented using Hadoop ecosystem components.**
- **The dataset was stored in HDFS, processed using MapReduce to calculate average movie ratings, and analyzed using Hive to extract meaningful insights such as top-rated movies.**
- **The project helped in understanding the working of distributed data processing and data analysis techniques.**
- **Generative AI acted as a smart assistant throughout the project by helping in code generation, debugging, and understanding complex concepts.**

❖ **Generative AI was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency.**

Ajay Chauhan

➢ Question-1:- Movie Rating Analysis using Hadoop Streaming.

❖ **Aim: - "To analyze movie dataset and calculate average rating using Hadoop MapReduce."**

Step-01: - Create **a "CSV" file.**

- **Cat > movie.csv**

```
[cloudera@quickstart ~]$ cat > movie.csv
userId,movieId,title,genre,rating,year,watch_time(min),device,location,review_score
1,101,KGF,Action,5,2018,150,Mobile,India,9
2,102,RRR,Action,4,2022,180,Laptop,India,8
3,103,Inception,Sci-Fi,5,2010,148,TV,USA,9
4,104,Avengers,Action,4,2012,143,Mobile,UK,8
5,105,Titanic,Romance,5,1997,195,TV,Canada,9
6,101,KGF,Action,3,2018,150,Laptop,India,7
7,102,RRR,Action,5,2022,180,TV,India,9
8,103,Inception,Sci-Fi,4,2010,148,Mobile,USA,8
9,104,Avengers,Action,5,2012,143,Laptop,UK,9
10,105,Titanic,Romance,4,1997,195,TV,Canada,8
11,106,Dangal,Drama,5,2016,161,Mobile,India,9
12,107,3 Idiots,Comedy,5,2009,170,Laptop,India,10
13,108,Interstellar,Sci-Fi,4,2014,169,TV,USA,9
14,109,Bahubali,Action,5,2015,159,Mobile,India,9
15,110,Sholay,Drama,5,1975,204,TV,India,10
16,106,Dangal,Drama,4,2016,161,Laptop,India,8
17,107,3 Idiots,Comedy,5,2009,170,TV,India,10
18,108,Interstellar,Sci-Fi,5,2014,169,Mobile,USA,10
19,109,Bahubali,Action,4,2015,159,Laptop,India,8
20,110,Sholay,Drama,5,1975,204,TV,India,10
21,101,KGF,Action,4,2018,150,Mobile,India,8
22,111,Dhurendhar,Action,5,2025,180,TV,India,9
23,103,Inception,Sci-Fi,5,2010,148,Laptop,USA,9
24,104,Avengers,Action,4,2012,143,Mobile,UK,8
25,105,Titanic,Romance,5,1997,195,TV,Canada,9
26,112,Deewaniyat,Romance,4,2024,160,Mobile,India,8
27,113,OG,Action,5,2024,170,Laptop,India,9
28,114,Saaiyara,Drama,4,2024,150,TV,India,8
29,115,Leo,Action,4,2023,165,Mobile,India,8
30,116,Salaar,Action,5,2023,175,Laptop,India,9
[cloudera@quickstart ~]$
```

Step-02:- Create a directory

- **hdfs dfs -mkdir /user/cloudera/ajaychauhan**
- **hdfs dfs -mkdir /user/cloudera/ajaychauhan/movie**

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/ajaychauhan
[cloudera@quickstart ~]$ hdfs dfs -mkdir /user/cloudera/ajaychauhan/movie
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/ajaychauhan/movie
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/ajaychauhan
Found 1 items
drwxr-xr-x   - cloudera cloudera          0 2026-04-20 08:39 /user/cloudera/ajay
chauhan/movie
[cloudera@quickstart ~]$
```

Step-03:- Load the "CSV" on Cloudera's Server

- **hdfs dfs -put movie.csv /user/cloudera/ajaychauhan/movie**
- **hdfs dfs -ls /user/cloudera/ajaychauhan/movie**

```
[cloudera@quickstart ~]$ hdfs dfs -put movie.csv /user/cloudera/ajaychauhan/movie
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/ajaychauhan/movie
Found 1 items
-rw-r--r--   1 cloudera cloudera       1455 2026-04-20 08:41 /user/cloudera/ajaychauhan/movie/movie.csv
[cloudera@quickstart ~]$
```

Ajay Chauhan

Step-04:- Create a mapper file using "nano"

- **nano mapper.py**

- **#!/usr/bin/env python**
- **import sys**
- **for line in sys.stdin:**
- **if "userId" in line:**
- **continue**
- **data = line.strip().split(',')**
- **movie = data[2]**
- **rating = float(data[4])**
- **print("%s\t%s" % (movie, rating))**

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    if "userId" in line:
        continue
    data = line.strip().split(',')
    movie = data[2]
    rating = float(data[4])
    print("%s\t%s" % (movie, rating))
```

Step-05:- Create a reducer file using "nano"

- **nano reducer.py**

```
#!/usr/bin/env python
import sys

current_movie = None
total = 0
count = 0

for line in sys.stdin:
    movie, rating = line.strip().split('\t')
    rating = float(rating)

    if current_movie == movie:
        total += rating
        count += 1
    else:
        if current_movie:
            print("%s\t%s" % (current_movie, total/count))
        current_movie = movie
        total = rating
        count = 1

if current_movie:
    print("%s\t%s" % (current_movie, total/count))
```

Step-06:- Make executable

- **chmod +x mapper.py reducer.py**

```
[cloudera@quickstart mapreduce]$ chmod +x mapper.py reducer.py
[cloudera@quickstart mapreduce]$ ls
mapper.py  reducer.py
[cloudera@quickstart mapreduce]$ ls -l
total 8
-rwxrwxr-x 1 cloudera cloudera 220 Apr 20 08:46 mapper.py
-rwxrwxr-x 1 cloudera cloudera 482 Apr 20 08:49 reducer.py
[cloudera@quickstart mapreduce]$
```

Step-07:- Copy "movie.csv" file, from /home/cloudera to /home/cloudera/mapreduce

- **cp /home/cloudera/movie.csv /home/cloudera/mapreduce/movie.csv**

```
[cloudera@quickstart mapreduce]$ cp /home/cloudera/movie.csv /home/cloudera/mapreduce/movie.csv
[cloudera@quickstart mapreduce]$ ls
mapper.py  movie.csv  reducer.py
[cloudera@quickstart mapreduce]$
```

Ajay Chauhan

Step-08:- Local test

- **cat movie.csv | python mapper.py**
- **cat movie.csv | python mapper.py | sort | python reducer.py**

```
[cloudera@quickstart mapreduce]$ cat movie.csv | python mapper.py
KGF     5.0
RRR     4.0
Inception       5.0
Avengers        4.0
Titanic 5.0
KGF     3.0
RRR     5.0
Inception       4.0
Avengers        5.0
Titanic 4.0
Dangal  5.0
3 Idiots        5.0
Interstellar    4.0
Bahubali        5.0
Sholay  5.0
Dangal  4.0
3 Idiots        5.0
Interstellar    5.0
Bahubali        4.0
Sholay  5.0
KGF     4.0
Dhurendhar      5.0
Inception       5.0
Avengers        4.0
Titanic 5.0
Deewaniyat      4.0
OG      5.0
Saaiyara        4.0
Leo     4.0
Salaar  5.0
[cloudera@quickstart mapreduce]$ cat movie.csv | python mapper.py | sort | python reducer.py
3 Idiots        5.0
Avengers        4.33333333333
Bahubali        4.5
Dangal  4.5
Deewaniyat      4.0
Dhurendhar      5.0
Inception       4.66666666667
Interstellar    4.5
KGF     4.0
Leo     4.0
OG      5.0
RRR     4.5
Saaiyara        4.0
Salaar  5.0
Sholay  5.0
Titanic 4.66666666667
[cloudera@quickstart mapreduce]$
```

Step-09:- Run Hadoop Job

- **hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py,reducer.py -input /user/cloudera/ajaychauhan/movie/movie.csv -output /user/cloudera/ajaychauhan/movie/output -mapper "python mapper.py" -reducer "python reducer.py"**

```
[cloudera@quickstart mapreduce]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py,reducer.p
y -input /user/cloudera/ajaychauhan/movie/movie.csv -output /user/cloudera/ajaychauhan/movie/output -mapper "python m
apper.py" -reducer "python reducer.py"
packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob7469730242140418706.jar tmpDir=null
26/04/20 08:58:31 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/20 08:58:31 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/20 08:58:33 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/20 08:58:33 INFO mapreduce.JobSubmitter: number of splits:2
26/04/20 08:58:33 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776698867558_0001
26/04/20 08:58:35 INFO impl.YarnClientImpl: Submitted application application_1776698867558_0001
26/04/20 08:58:35 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_177
6698867558_0001/
26/04/20 08:58:35 INFO mapreduce.Job: Running job: job_1776698867558_0001
26/04/20 08:58:51 INFO mapreduce.Job: Job job_1776698867558_0001 running in uber mode : false
26/04/20 08:58:51 INFO mapreduce.Job:  map 0% reduce 0%
26/04/20 08:59:02 INFO mapreduce.Job:  map 50% reduce 0%
26/04/20 08:59:03 INFO mapreduce.Job:  map 100% reduce 0%
26/04/20 08:59:15 INFO mapreduce.Job:  map 100% reduce 100%
26/04/20 08:59:16 INFO mapreduce.Job: Job job_1776698867558_0001 completed successfully
26/04/20 08:59:16 INFO mapreduce.Job: Counters: 49
```

Step-10:- View Output

- **hdfs dfs -cat /user/cloudera/ajaychauhan/movie/output/part-00000**

```
[cloudera@quickstart mapreduce]$ hdfs dfs -ls /user/cloudera/ajaychauhan/movie/output
Found 2 items
-rw-r--r--   1 cloudera cloudera          0 2026-04-20 08:59 /user/cloudera/ajaychauhan/movie/output/_SUCCESS
-rw-r--r--   1 cloudera cloudera        219 2026-04-20 08:59 /user/cloudera/ajaychauhan/movie/output/part-00000
[cloudera@quickstart mapreduce]$ hdfs dfs -cat /user/cloudera/ajaychauhan/movie/output/part-00000
3 Idiots        5.0
Avengers        4.33333333333
Bahubali        4.5
Dangal  4.5
Deewaniyat      4.0
Dhurendhar      5.0
Inception       4.66666666667
Interstellar    4.5
KGF     4.0
Leo     4.0
OG      5.0
RRR     4.5
Saaiyara        4.0
Salaar  5.0
Sholay  5.0
Titanic 4.66666666667
[cloudera@quickstart mapreduce]$
```

- **<u>Question-2:-</u> To perform data analysis on processed movie dataset using Hive.**

  - **Aim: - "To analyze movie data and find top-rated movies using Hive queries."**

Step-01:- Open Hive

- **hive**

```
[cloudera@quickstart mapreduce]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02:- Create table on hive

- **CREATE TABLE movie_avg (**
  movie STRING,
  avg_rating FLOAT
)
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY '\t';

```
hive> CREATE TABLE movie_avg (
    >  movie STRING,
    >  avg_rating FLOAT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t';
OK
Time taken: 4.05 seconds
hive> show tables;
OK
count_string
movie_avg
Time taken: 0.382 seconds, Fetched: 2 row(s)
hive>
```

Step-03:- Load data on the table in movie_avg

- **LOAD DATA INPATH '/user/cloudera/ajaychauhan/movie/output' INTO TABLE movie_avg;**

```
hive> LOAD DATA INPATH '/user/cloudera/ajaychauhan/movie/output' INTO TABLE movie_avg;
Loading data to table default.movie_avg
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/movie_avg/part-00000': User does no
t belong to hive
Table default.movie_avg stats: [numFiles=1, totalSize=219]
OK
Time taken: 1.471 seconds
hive>
```

Step-04:- View Data

- **Select * from movie_avg;**

```
hive> SELECT * FROM movie_avg;
OK
3 Idiots        5.0
Avengers        4.3333335
Bahubali        4.5
Dangal  4.5
Deewaniyat      4.0
Dhurendhar      5.0
Inception       4.6666665
Interstellar    4.5
KGF     4.0
Leo     4.0
OG      5.0
RRR     4.5
Saaiyara        4.0
Salaar  5.0
Sholay  5.0
Titanic 4.6666665
Time taken: 0.857 seconds, Fetched: 16 row(s)
hive>
```

Step-05:- Find Top Movies

- **Select * from movie_avg order by avg_rating desc;**

```
hive> SELECT * FROM movie_avg ORDER BY avg_rating DESC;
Query ID = cloudera_20260420090909_04fba5c2-85cf-43ea-aea4-2b7756c7d776
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776698867558_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776698867558
_0002/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776698867558_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-20 09:10:09,363 Stage-1 map = 0%,  reduce = 0%
2026-04-20 09:10:20,037 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 3.05 sec
2026-04-20 09:10:31,713 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 6.39 sec
MapReduce Total cumulative CPU time: 6 seconds 390 msec
Ended Job = job_1776698867558_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 6.39 sec   HDFS Read: 5822 HDFS Write: 207 SUCCESS
Total MapReduce CPU Time Spent: 6 seconds 390 msec
OK
OG      5.0
Sholay  5.0
Salaar  5.0
Dhurendhar      5.0
3 Idiots        5.0
Titanic 4.6666665
Inception       4.6666665
RRR     4.5
Interstellar    4.5
Dangal  4.5
Bahubali        4.5
Avengers        4.3333335
KGF     4.0
Saaiyara        4.0
Deewaniyat      4.0
Leo     4.0
Time taken: 40.217 seconds, Fetched: 16 row(s)
hive>
```

Step-06:- Top 5 Movies

- **Select * from movie_avg order by avg_rating desc limit 5;**

```
hive> SELECT * FROM movie_avg ORDER BY avg_rating DESC LIMIT 5;
Query ID = cloudera_20260420091212_47c4e033-f57b-415a-9f23-cca2fe7d76d5
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1776698867558_0003, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1776698867558
_0003/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1776698867558_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-20 09:13:05,391 Stage-1 map = 0%,  reduce = 0%
2026-04-20 09:13:14,846 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.05 sec
2026-04-20 09:13:26,308 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 5.01 sec
MapReduce Total cumulative CPU time: 5 seconds 10 msec
Ended Job = job_1776698867558_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 5.01 sec   HDFS Read: 5961 HDFS Write: 57 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 10 msec
OK
OG      5.0
Sholay  5.0
Salaar  5.0
Dhurendhar      5.0
3 Idiots        5.0
Time taken: 35.365 seconds, Fetched: 5 row(s)
hive>
```

- **Output:**

  - **Hive successfully displayed:**
    - **Average ratings**
    - **Top-rated movies**

- **In this practical, Hive was used to analyze processed movie data and retrieve meaningful insights using SQL-like queries.**